A Contract-Centered Architecture for Scalable and Manageable Agentic Runtimes

Yaxiao Liu
rootliu@gmail.com

Working preprint | 2026-08-08

1. Abstract

Agentic runtimes face a systems question that task benchmarks do not answer: whether independently deployable behavior and physical execution capacity can change on separate schedules without hidden coupling or prohibitive enforcement cost. A product owner wants to activate a new behavior without redesigning the execution fleet; a platform owner wants to add workers, isolation zones, or accelerators without changing what an admitted behavior means. One research question organizes the paper: can a runtime activate independently deployable capabilities without materially changing capacity response, while scaling capacity without materially changing capability semantics, at an acceptable enforcement cost?

We state one primary hypothesis, P1: within a declared operating region, changing activated capability should preserve the capacity-response interaction within a preregistered two-sided equivalence margin, while changing compatible Scaffold capacity should preserve capability semantics up to a preregistered directional non-inferiority margin. A Harness provides logical admission, path construction, policy mediation, and evidence capture; a Scaffold provides physical execution, isolation, and resource control.

We propose a cluster-period randomized crossover experiment with balanced order, reset or washout, repeated seeds and failure regimes, and cluster-aware uncertainty. The design measures the capability-by-Scaffold interaction, semantic outcomes, six mechanism obligations, and enforcement overhead against a declared budget. This paper supplies a responsibility model and a falsifiable measurement protocol. It reports no completed runtime implementation, experiment, dataset, or measured result.

2. Introduction

An agentic runtime must answer two different change requests. A product owner wants to activate a new behavior without redesigning the execution fleet. A platform owner wants to add workers, isolation zones, accelerators, or regional capacity without changing what an admitted behavior means. Current architecture descriptions often name both concerns but evaluate them separately: capability work is judged by task success, while infrastructure work is judged by throughput and latency. Those one-axis evaluations cannot reveal whether the axes recouple through shared state, resource-sensitive inputs, scheduler feedback, undeclared effects, or bypasses around policy enforcement.

The distinction matters because a positive capacity effect is desirable. Adding compatible Scaffold resources should normally improve throughput or queueing delay. The scientific question is not whether the Scaffold has a zero main effect. It is whether the response to a capability intervention materially changes across Scaffold configurations, whether semantic outcomes remain non-inferior, and whether the controls required to establish that result stay within an acceptable enforcement budget.

This paper asks one research question:

Can a runtime activate independently deployable capabilities without materially changing capacity response, while scaling capacity without materially changing capability semantics, at an acceptable enforcement cost?

We call the bounded claim cost-aware capability-capacity separability. It is neither universal modularity nor proof that a named implementation scales. It is a three-way empirical decision about a specified runtime, workload, policy, data snapshot, and compatible capacity range. The result can be supported within Omega, falsified within Omega, or inconclusive.

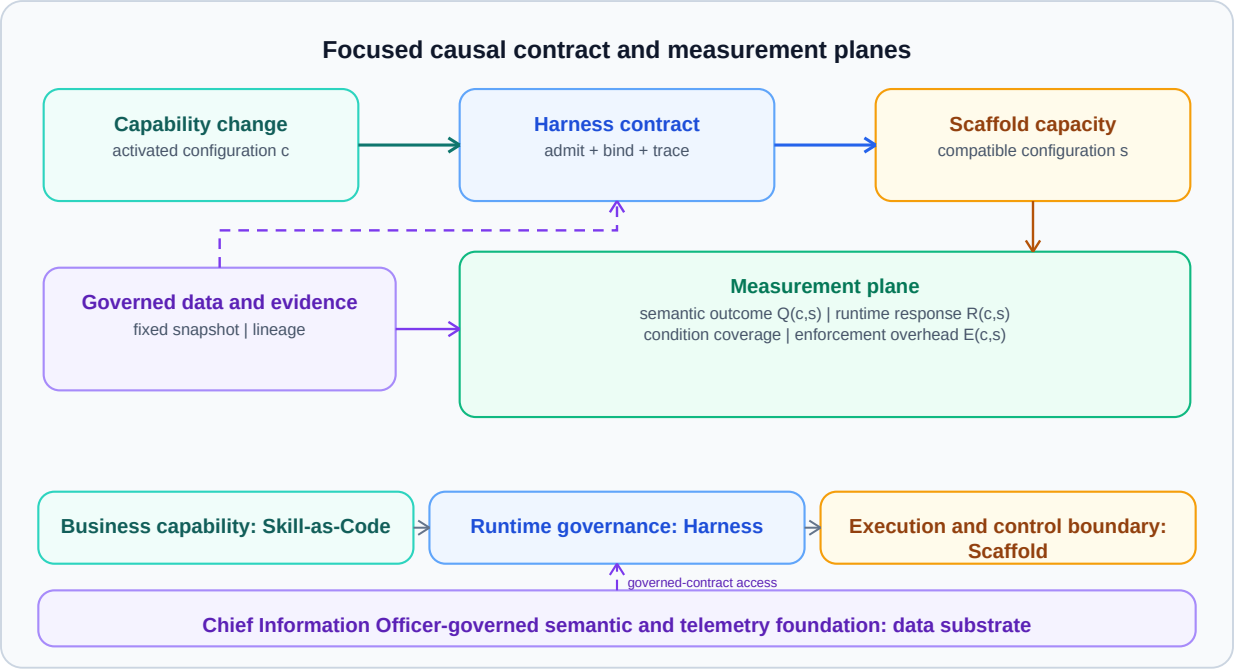

Figure 1. *Shown: capability change enters through an activated Skill bundle, the Harness resolves its contract, Scaffold capacity supplies execution, and governed data plus the measurement plane remain explicit. Why it matters: the diagram separates the two interventions and identifies the boundary at which coupling and enforcement cost must be observed. Class: architecture.*

Figure 1 is the paper's organizing view. Capability growth means activated independently deployable behavior on admitted paths. It does not mean the number of entries in a registry. Capacity growth means a change to compatible physical execution resources under fixed logical policy and fixed data snapshots. The Harness is the binding boundary between those interventions.

The contribution is narrow. First, we define responsibility and observability boundaries for Skill, Harness, Scaffold, and an external data substrate. Second, we turn six familiar design conditions into measured obligations whose coverage, violations, uncertainty, cost, and exclusions determine whether P1 is even decidable. Third, we specify a cluster-period crossover that estimates interaction, semantic non-inferiority, and enforcement overhead without treating an expected Scaffold main effect as evidence against separability.

3. Responsibility Model and Primary Claim

3.1 Four Responsibility Objects

Skill.

A Skill is a versioned, independently deployable behavior declaration. It owns task intent, typed inputs and outputs, permitted effects, preconditions, tests, and release identity. A Skill may package instructions, tool adapters, executable code, examples, and verifier rules, but it does not choose physical placement, grant itself credentials, or redefine platform isolation. The unit used in this paper is an activated Skill bundle on an admitted path, not a passive registry entry.

Harness.

The Harness compiles a request and an activated bundle into a resolved execution contract. Harness owns logical admission and control. That ownership includes path construction, schema checking, policy evaluation, effect authorization, logical budgets, binding constraints, version identities, and evidence obligations. Planning may be probabilistic; the decision to admit a fixed resolved contract under a fixed policy snapshot must be replayable.

Scaffold.

The Scaffold is the physical execution and resource-control boundary. Scaffold owns physical execution and isolation. It supplies compute, process or container boundaries, network and data locality, runtime identity, resource accounting, queueing, failure containment, and attestation. It exposes resource facts to the Harness but does not reinterpret task goals or silently add capability semantics.

External data substrate.

The external data substrate owns source authority, semantic definitions, snapshots, lineage, credentials, retention, and policy-relevant data facts. It is not a hidden Scaffold database and not a Skill-owned cache. The Harness resolves authorized access and evidence requirements; the Scaffold enforces the resulting locality and network constraints.

These are responsibility boundaries, not mandatory teams or services. One implementation can combine objects, but its measurements must still distinguish logical admission, physical execution, and data authority. Information hiding is useful only when the hidden decisions have explicit interfaces and ownership [1].

3.2 Canonical Configurations and Outcomes

The activated capability configuration c identifies the exact versioned Skill bundle, admitted graph, tool and effect surface, model settings, and verifier set exercised by the workload. The Scaffold capacity configuration s identifies the compatible worker count, resource class, isolation topology, placement constraints, and queueing limits. Capability is denoted by c; the symbol s is reserved for Scaffold capacity and is never used for Skill.

The declared operating region Omega fixes the workload family and mix, tenant and identity model, Harness and policy versions, model and tokenizer versions, external-service contracts, data snapshots, compatible Scaffold classes, scheduler regime, and failure regimes. A result outside those bounds is a new study, not an extrapolation.

For each assigned cluster-period we observe three named quantities:

- runtime response R(c,s): throughput, admission rate, queueing delay, latency distribution, saturation, retries, and cost per completed run;
- semantic outcome Q(c,s): task success, every-requirement satisfaction, output schema adherence, authorized-effect adherence, and postcondition satisfaction; and
- enforcement overhead E(c,s): added latency, compute, memory, control-plane work, evidence volume, and monetary cost attributable to admission, mediation, isolation, and measurement.

The literal names runtime response R(c,s), semantic outcome Q(c,s), and enforcement overhead E(c,s) are part of the measurement contract. They prevent a favorable throughput result from substituting for semantic stability or a favorable semantic result from hiding an unaffordable control path.

3.3 P1: Cost-Aware Separability

Let c_0 and c_1 be preregistered capability configurations and let s_0 and s_1 be compatible Scaffold capacity configurations. The primary runtime endpoint $Y_R(c,s)$ is the natural logarithm of period-level p95 admission-to-terminal latency in seconds. Every admitted request is included; a request without a terminal event by the preregistered timeout tau is assigned latency tau. Because requests assigned tau form a point mass at the censoring bound, and that mass can dominate the period-level p95 when timeout rates differ across arms, the report must also give the per-cell timeout rate and run a sensitivity analysis on an alternative endpoint (for example, the p95 of completed runs or a winsorized

quantile); the equivalence decision on the primary endpoint must be robust to the recorded timeout distribution. The primary difference-in-differences estimand is

$$Delta_R = [Y_R(c_1,s_1)-Y_R(c_0,s_1)] -[Y_R(c_1,s_0)-Y_R(c_0,s_0)].$$

The two-sided equivalence margin is $m_R=\log(1.10)$, a 1.10-fold bound on the ratio of latency ratios: equivalently, an interaction of at most a 10% multiplicative change in p95 tail latency. The runtime criterion is satisfied only when the 90% two-sided cluster-aware confidence interval for $Delta_R$ lies wholly inside $[-m_R,m_R]$, the two one sided tests (TOST) rule at level 0.05 [2]. This is an equivalence test, not a directional non-inferiority test. For more than two levels, the same named endpoint, scale, margin, and interval rule apply to every preregistered capability-by-Scaffold interaction contrast in a simultaneous cluster-aware model. The phrase capability-by-Scaffold interaction estimand refers to this departure of the capability contrast across capacity settings. The other components of R(c,s) remain reported secondary endpoints and cannot replace Y_R after outcomes are observed.

P1 does not constrain the Scaffold main effect $Y_R(c,s_1)-Y_R(c,s_0)$. A positive Scaffold main effect on throughput, or a negative main effect on latency, is expected and does not falsify P1. The primary semantic endpoint $Q_{req}(c,s)$ is the proportion of admitted runs satisfying every preregistered requirement. At each capability level, the capacity contrast is $D_Q(c)=Q_{req}(c,s_1)-Q_{req}(c,s_0)$. Semantic stability uses directional non-inferiority: with $m_Q=0.05$, the one-sided 95% cluster-aware lower confidence bound for every required $D_Q(c)$ must exceed -0.05 [3]. Other semantic outcomes are secondary unless a separate scale, direction, margin, and multiplicity rule is preregistered.

The enforcement budget B_E is fixed before outcomes are examined and applies to the vector of overhead measures: every mandatory component of E(c,s) must remain within its declared latency, compute, evidence-volume, and monetary limits under the paired counterfactual in Section 7. A cheap uninstrumented path cannot satisfy this budget because missing control work makes the evidence ineligible.

P1. Within the declared operating region mega, an admitted capability intervention preserves the log-p95 latency capacity-response relationship under the two-sided interaction-equivalence rule, a compatible Scaffold intervention preserves every-requirement satisfaction under the directional non-inferiority rule, and the measured controls remain within B_E.

The decision rule has three states.

- supported within Omega: all six obligations in Section 6 pass coverage, calibrated-sensitivity, and worst-case violation-bound gates; the 90% interaction interval lies inside $[-m_R,m_R]$; every semantic lower bound exceeds $-m_Q$; and every enforcement-overhead upper bound lies below its component of B_E;
- falsified within Omega: eligibility and calibration are sufficient for a decision, and the interaction interval lies wholly above m_R or below $-m_R$, a semantic upper bound lies below $-m_Q$, an observed-violation lower bound exceeds its ceiling, or an enforcement-overhead lower bound exceeds its budget; and
- inconclusive: instrumentation coverage or monitor calibration is insufficient, an interaction, semantic, violation, or overhead interval does not resolve its margin, crossover contamination remains after reset, a predeclared eligibility rule fails, or another required quantity cannot be estimated with the planned cluster-aware uncertainty.

The three-way rule distinguishes failure of the claim from failure to measure it. In particular, a nonsignificant interaction from an underpowered design is inconclusive, not support.

4. Foundations and Novelty Boundary

Reference monitors and least authority.

The proposed mechanisms are established systems ideas. A reference monitor must be tamper resistant, always invoked, and small enough to analyze; complete mediation requires checking every access rather than only the first one [4, 5]. Capability systems and least authority explain why an admitted path should receive narrow, unforgeable authority

instead of ambient privilege [6, 5]. Information hiding separates decisions likely to change behind stable interfaces [1], while Hydra made policy-mechanism separation an explicit operating-system design principle [7]. Control/data-plane separation similarly distinguishes decisions about forwarding or placement from execution of accepted work [8]. Effect non-interference has a longer history in security semantics: the obligation that one admitted run cannot change the authorized effects or semantic inputs of another is a form of non-interference [9].

Control loops, quality models, and queueing.

Autonomic-computing work organizes monitoring, analysis, planning, and execution around shared knowledge; Monitor, Analyze, Plan, Execute over a shared Knowledge base (MAPE-K) is relevant here as a control-loop vocabulary, not as evidence that the loop is correct [10, 11]. ISO/IEC 25010 supplies a product-quality vocabulary for performance efficiency, reliability, security, maintainability, and portability, but it does not select margins for this runtime [12]. Queueing theory predicts nonlinear response near saturation, and datacenter studies show that shared-resource interference and tail latency can dominate aggregate averages [13, 14, 15]. Those results motivate period design, tail metrics, and resource instrumentation.

Contemporary agentic-runtime work.

Recent agentic-runtime work argues for explicit governance planes and path-level composition checks [16, 17]. Release-gate work demonstrates that machine-checkable invariants can be tested across capability changes, but it does not estimate a capability-by-capacity interaction or semantic non-inferiority across compatible capacity settings [18]. Skill lifecycle work supports treating behavior packages as persistent versioned artifacts, not as transient prompt fragments [19].

Novelty boundary.

Our contribution is therefore not a new reference monitor, capability model, modularity principle, scheduler, quality taxonomy, or autonomic loop. It composes established mechanisms into an agentic-runtime responsibility model and makes separability plus enforcement cost falsifiable. The empirical object is the joint behavior of activated capability configuration c and Scaffold capacity configuration s inside a declared operating region Omega.

5. Contract Boundary and Architecture

5.1 Resolved Harness Contract

For an admitted run, the Harness emits a resolved contract

$$C = < I, O, G, A, B, V >,$$

where I and O are typed inputs and outputs, G is the activated graph, A is the authority and effect set, B contains budgets and binding constraints, and V pins policy, model, data, capability, verifier, and trace versions. Each field is serializable and independently rejectable. A run rejected at admission produces a reason and no external effect. An accepted run carries the contract identity through execution and evidence.

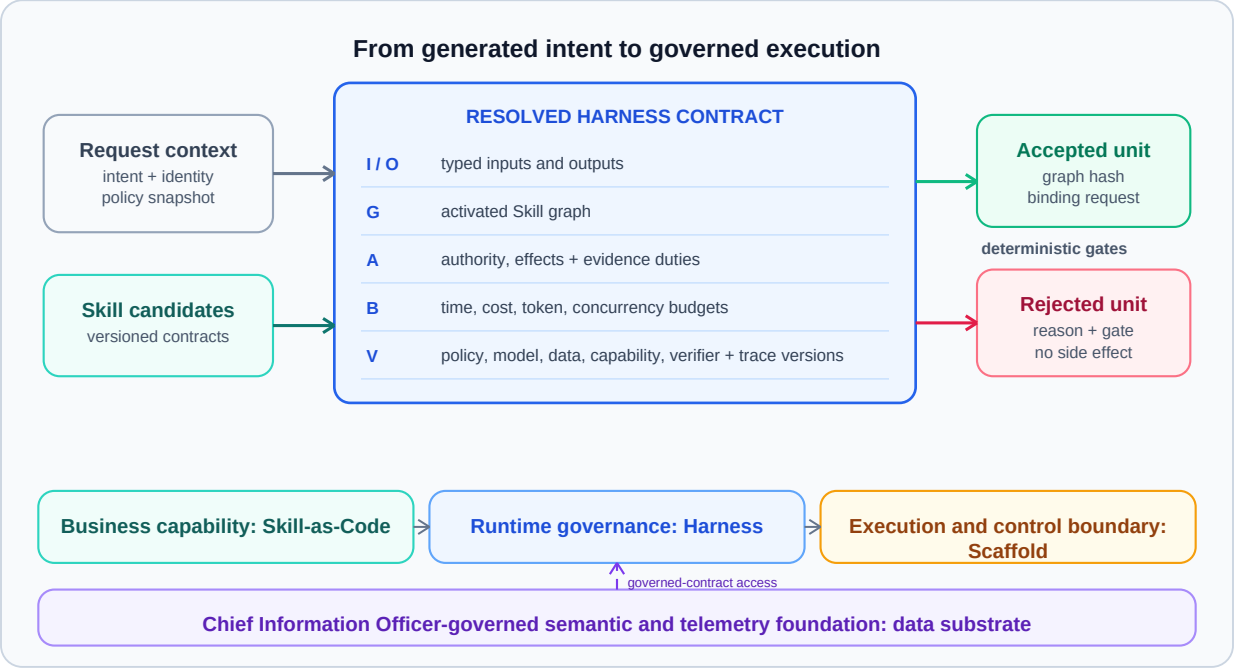

Figure 2. *Shown: a request and activated bundle pass deterministic admission gates to become a resolved Harness contract with typed data, path, authority, budgets, versions, binding, and trace identity. Why it matters: the contract is the inspectable unit on which complete mediation, replay, and capability-to-capacity binding are tested. Class: architecture.*

The Harness contract makes selective activation observable. The registry can contain many Skills, but c changes only when activated behavior on an admitted path changes. This prevents registry cardinality from being mistaken for a capability intervention and gives the experiment a stable treatment identity.

5.2 Logical Control and Physical Execution

Control decisions include activation, graph validation, authorization, policy evaluation, budget assignment, and placement constraints. Physical execution begins only after admission and includes queueing, resource allocation, isolation, network access, effect execution, and failure containment. Evidence spans both: every logical decision and physical action carries the contract and period identity.

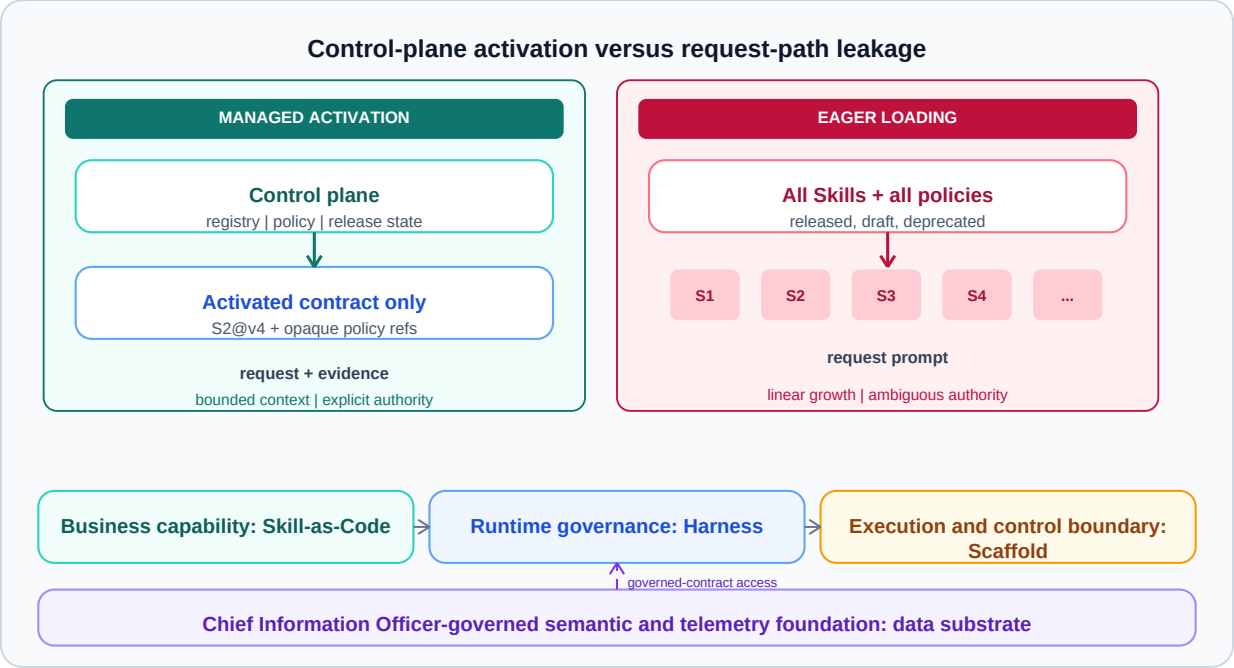

Figure 3. *Shown: the control plane holds the registry, policy, admission, and placement logic, while the data plane receives only the resolved contract and executes accepted work. Why it matters: keeping inactive registry state and mutable control metadata off the request path limits unmeasured coupling between capability growth and runtime load. Class: architecture.*

Separation is not established by drawing two planes. The complete-mediation and scheduler-independence obligations must observe every crossing. A cache fill, quota check, retry controller, or placement hint that bypasses the resolved contract remains a possible coupling channel.

5.3 Bounded Composition

When a running path discovers a need outside its admitted tool or data authority, the current agent does not acquire that authority inline. It returns a typed unmet dependency. The Harness may resolve a new bounded sub-agent contract, admit it as a child, and require a verifier or semantic join before its result can satisfy the parent. Termination depends on declared postconditions rather than an unconstrained agent judgment.

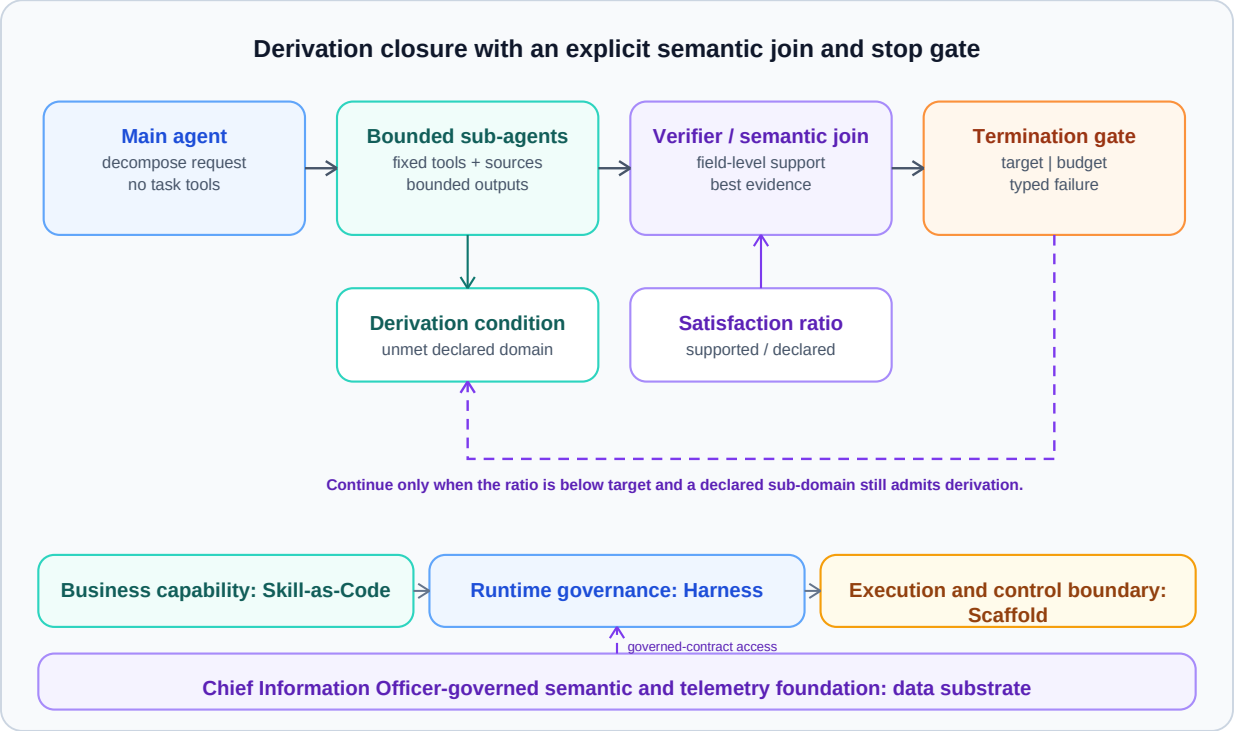

Figure 4. *Shown: a main agent derives bounded sub-agents for unmet dependencies, then a separate verifier or semantic join checks their returned evidence before a termination gate closes the path. Why it matters: capability expansion during a run remains a sequence of admitted contracts rather than an invisible increase in ambient authority. Class: protocol.*

This protocol is secondary to P1. It offers one way to preserve typed closure under dynamic composition, but P1 requires only that the implemented composition path be instrumented and judged under the six obligations.

5.4 External Data Resolution

A Skill names required semantic fields and admissible source classes. The data authority resolves those requirements against a versioned contract. The Harness binds an authorized query and evidence obligation; the Scaffold executes the fetch inside the required identity, network, and locality boundary. Returned data carries source, snapshot, transformation, and access-decision identities.

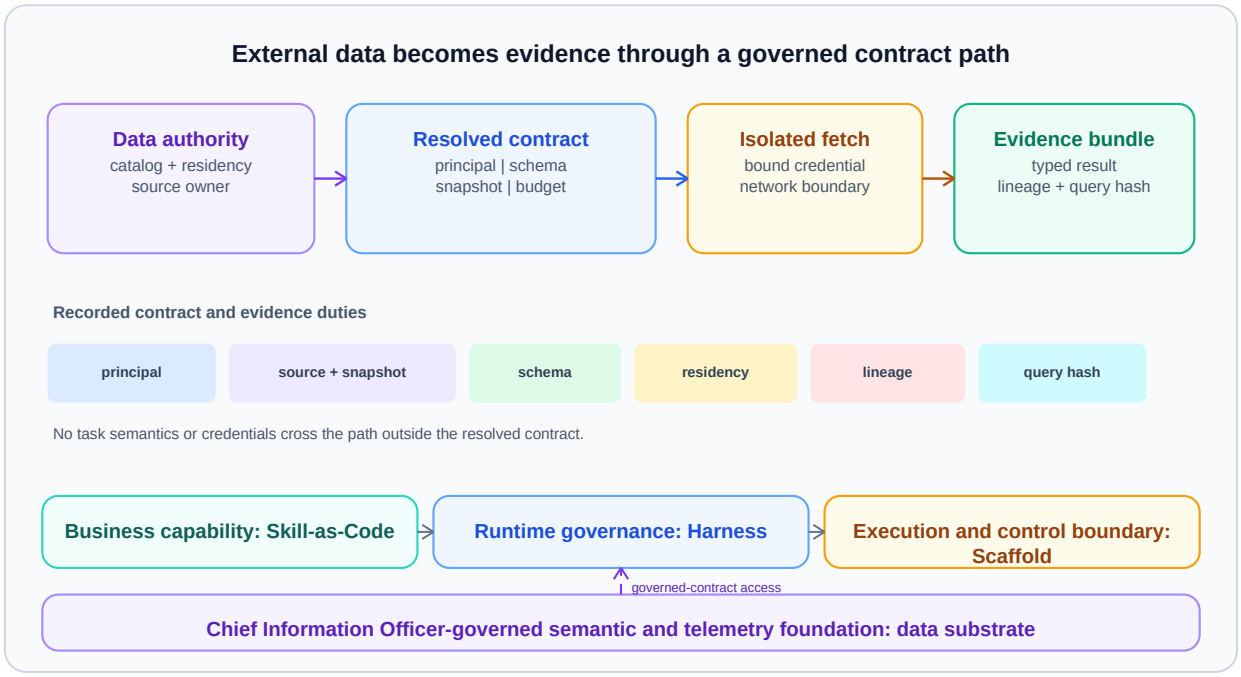

Figure 5. *Shown: data authority resolves a semantic request into a governed contract, the Scaffold performs an isolated fetch, and the run receives an evidence bundle rather than source credentials. Why it matters: data policy and snapshot identity remain fixed experimental inputs instead of hidden properties of the capability or capacity treatment. Class: architecture.*

The external substrate can change independently only outside an active study. Within Omega, the data snapshot and policy-relevant facts are fixed or explicitly assigned as a failure regime. A live source whose contents drift without version identity makes semantic non-inferiority inconclusive.

5.5 Placement, Isolation, and Evidence

The Harness can dry-run a resolved graph to reject impossible authority, budget, locality, or dependency combinations before effects occur. It then binds accepted nodes to compatible Scaffold resources. The Scaffold enforces the binding and emits resource, isolation, queue, retry, and effect records. The evidence plane joins these records by contract, cluster, period, and workload identity.

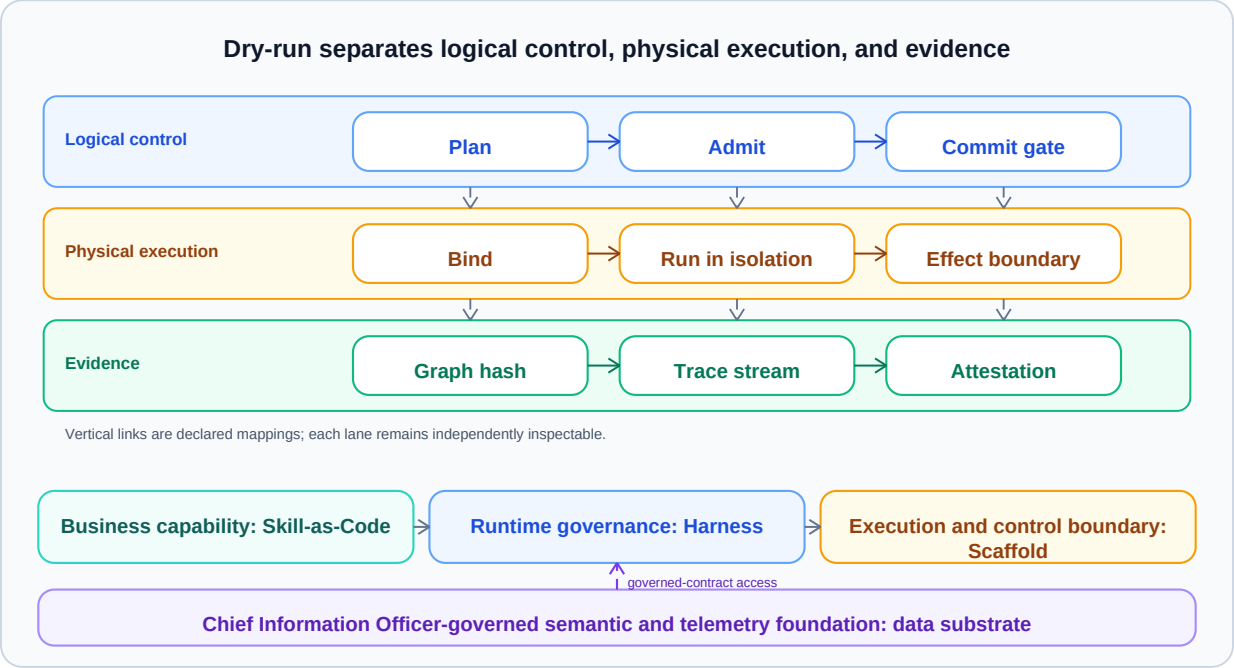

Figure 6. *Shown: logical control validates and binds a graph, physical execution runs it in compatible isolation and locality zones, and an evidence path records both decisions and effects. Why it matters: placement optimization remains distinguishable from semantic policy while still exposing the resource channels that could recouple c and s. Class: architecture.*

Dry-run is not assumed to be beneficial. Its latency and avoided-work value are part of enforcement overhead E(c,s). A study can use immediate admission instead, but it must still record equivalent binding and rejection facts.

5.6 Versioned Skill Lifecycle

Skill release follows draft, validation, staged activation, monitored release, rollback, and retirement states. Promotion pins contract tests and verifier versions. Model, tool, or policy changes trigger revalidation when they alter the declared bundle. A rollback preserves identity and evidence rather than overwriting the released artifact.

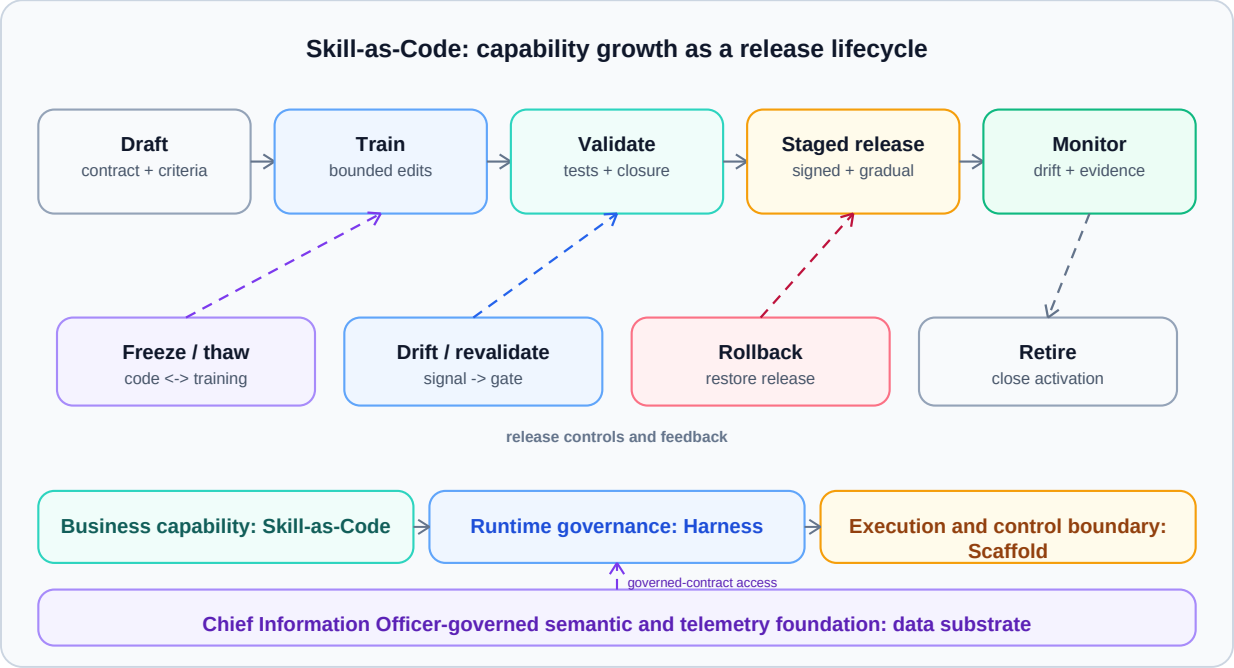

Figure 7. *Shown: a Skill moves through versioned validation, staged activation, monitored release, rollback, and retirement with evidence attached to each transition. Why it matters: the experiment can identify capability configuration c precisely and can repeat or reverse an intervention without conflating it with registry churn. Class: protocol.*

Training methods, automated repair, and cross-registry knowledge structures are outside the primary claim. They can propose a new Skill version, but they do not bypass validation or enter a P1 period as an unrecorded mutation.

5.7 Threat Model

The Harness is a reference monitor, so what it protects and against whom must be stated before the six obligations can be read. We assume the following, and each assumption is a place the architecture can be attacked rather than a property it establishes.

- Trusted computing base. The control plane is trusted: the Harness compiler, the Skill registry, the policy evaluator, the gate implementations, and the Scaffold attestation service. A compromise of any of them defeats the boundary, which is why the design requires replicated control services, signed artifacts, and independent attestation, and degrades by reducing capability rather than widening governance.
- Untrusted model output. Planner and executor output is an untrusted request for authority, never an authorization. A plan that names an operation outside the resolved contract is a rejected request, not a widened contract; this is why the admission decision is deterministic for a fixed contract and policy snapshot while plan generation is not.
- Semi-trusted Skills. A Skill author is authenticated and accountable but not assumed correct or benign. A Skill may declare one behavior and exhibit another, so declarations are admission inputs to be validated behaviorally, and operation closure is a gate rather than a claim.
- Untrusted external content. Tool results, retrieved documents, and data-source payloads are attacker-influenceable. Text that arrives through the data plane cannot change the activated path, the authorized effect set, or a policy version, because those are resolved in the control plane before the fetch occurs; contract resolution, rather than instruction filtering, is the response to prompt injection.
- Adversary capabilities. The adversary can author or modify a Skill, influence request text, control external content, and compose approved Skills into a hazardous path. The adversary cannot break Scaffold isolation, forge attestation, or subvert the control plane; those are Scaffold and platform security properties assumed rather than argued here.

The design goal is narrow: every effect an adversary can reach must appear in some resolved contract's authorized effect set, and every attempt outside that set must be a typed failure recorded in the trace. Gates and traces make a violation attributable, and complete mediation makes an undeclared channel detectable, but neither prevents an authorized effect from being harmful.

6. Six Measured Obligations

The six conditions below are not architectural virtues awarded by inspection. Each is a measured obligation with five required outputs: instrumentation coverage, observed violations, uncertainty, enforcement cost, and operating-region exclusions. For obligation j, an independent inventory supplies the eligible-event count N_j; the monitor supplies O_j joinable event records and observes X_j violations among them. Uncovered events are $M_j = N_j - O_j$. Self-reported Harness traces cannot establish their own completeness.

Each obligation declares a coverage floor gamma_j, violation ceiling nu_j, and monitor-sensitivity floor eta_j before collection. Coverage must satisfy O_j/N_j at least gamma_j. For support, every uncovered event is treated as a violation: the one-sided 95% cluster-aware upper confidence bound U_j for the worst-case rate $(X_j+M_j)/N_j$ must be below nu_j for every obligation. Bounds are not pooled across obligations. Because the decision rule also resolves the interaction-equivalence and semantic non-inferiority margins, the analysis plan must preregister a single family-wise error rule - for example, joint coverage supplied by a simultaneous confidence region, or a closed testing sequence fixing the order in which obligations and margins are decided - before collection; the per-obligation bounds are otherwise not jointly valid. If the denominator is unavailable, coverage falls below its floor, or U_j does not clear its ceiling, the obligation cannot support P1; an independently bounded but unresolved missing-event count remains in M_j rather than being dropped.

Calibration is also mandatory. For every obligation, blinded diagnostic injections open preregistered known violation channels across the assigned cluster-periods. If K_j violations are injected and D_j are detected, the one-sided 95% cluster-aware lower confidence bound for sensitivity D_j/K_j must exceed eta_j. Injected events are labelled and excluded from the experimental violation numerator and semantic/runtime outcomes. Missing injections, a failed sensitivity floor, or a monitor that cannot be calibrated makes P1 inconclusive; high nominal coverage cannot substitute for calibration. The report includes all inventory, detection, injection, uncertainty, cost, and exclusion counts.

Typed closure.

Instrumentation enumerates every node, input, output, tool, data source, authority, and external effect on each activated path and joins those events to the resolved contract. A violation is an executed or requested operation absent from the typed bundle, including a dynamically derived node that was not separately admitted. Coverage compares contract manifests with independent tool, network, and effector inventories. Uncertainty is reported for undeclared operations per activated path and per operation. Validation latency and manifest/evidence volume enter E(c,s). Paths containing opaque executors whose operation inventory cannot be observed are excluded from Omega before outcomes are inspected.

Complete mediation.

Instrumentation sits at every effectful boundary: tool dispatch, credential issuance, network egress, persistent write, message send, and privileged runtime transition. A violation is an invocation that reaches such a boundary without a current resolved-contract authorization. Coverage is the fraction of independently observed boundary events matched to an admission record. The violation interval is cluster-aware because bypasses can share a process, node, or period. Interposition latency and controller capacity enter E(c,s). Effectors that cannot expose an independent call inventory are pre-outcome exclusions.

Effect non-interference.

Instrumentation labels writes, messages, cache mutations, model-side state, and externally visible effects with contract and tenant identities, then records readers and downstream consumers. A violation is an undeclared path by which one admitted run changes the authorized effects or semantic inputs of another. Coverage includes every mutable or communicative channel in the threat model, not only database writes. Uncertainty is reported by channel and

cluster-period. Labeling, taint or provenance processing, and isolation cost enter E(c,s). Shared services without attributable mutation or consumption logs are excluded from Omega.

Shared-state isolation.

Instrumentation records every read, write, lock, cache access, session update, and durable checkpoint against an assigned state partition. A violation is an access outside the contract's partition or an undeclared contention edge between treatment units. Coverage uses storage, cache, lock-manager, and process-level inventories. The report gives interval estimates for unauthorized accesses and cross-partition contention. Namespace, copy, flush, and reset costs enter E(c,s). State that cannot be reset, snapshotted, partitioned, or audited is excluded before period assignment.

Resource invariance.

Instrumentation hashes the semantic inputs visible to the activated behavior, including model settings, token limits, tool schemas, policy facts, data snapshot, clocks where relevant, and resource metadata exposed to the model or tools. A violation occurs when changing compatible s changes an undeclared semantic input or crosses a preregistered resource boundary that defines a different capability environment. Coverage is the fraction of semantic-input fields independently captured in every arm. Field-level differences and their uncertainty are reported, not averaged into one score. Capture and normalization costs enter E(c,s). Accelerator classes, memory limits, or degradation modes known to alter semantics are separate strata or outside Omega.

Scheduler independence.

Instrumentation records queue state, priorities, admission feedback, retry decisions, autoscaling signals, placement hints, cancellation, and scheduler-written metadata visible to the Harness, model, tools, or data path. A violation is an undeclared scheduler-to-semantics or capability-to-placement feedback edge. Coverage joins scheduler decisions to every admitted and rejected run. Violation intervals are estimated by cluster-period and failure regime. Scheduler logging, policy evaluation, and any traffic-shaping overhead enter E(c,s). Adaptive schedulers whose policy version or feedback channels cannot be pinned and observed are excluded from Omega.

Passing all six thresholds does not prove that no seventh coupling channel exists. It establishes that the preregistered threat model was measured well enough to apply P1 within Omega. The required diagnostic injections test the monitor assigned to each declared channel; they do not expand the claim beyond that threat model.

7. Cluster-Period Crossover Study

7.1 Unit, Assignment, and Interventions

The experimental unit is a cluster-period, also called a system epoch: an isolated worker pool, tenant partition, or deployment cell operated under one assigned (c,s) pair for a fixed period. Requests inside a period are repeated observations, not independent randomized units. Cluster-period assignment avoids pretending that requests sharing caches, queues, schedulers, or controllers are independent.

Clusters cross over through all eligible capability and Scaffold configurations. Period assignment is randomized within prespecified cluster and failure-regime blocks. Order balancing uses counterbalanced sequences so each treatment appears comparably early and late and follows every other treatment often enough to estimate differential carryover. The design follows established crossover and cluster-crossover principles [20, 21]. Period and sequence terms are fixed before outcome inspection, but neither term substitutes for an explicit lagged-treatment carryover estimate.

A capability intervention changes activated capability configuration c by activating a versioned bundle that occurs on admitted paths in the fixed workload. Adding inactive registry entries is not a treatment. A capacity intervention changes Scaffold capacity configuration s by altering compatible worker count, resource class, or topology while logical admission policy, activated bundle, model settings, workload mix, and data snapshots remain fixed. Both interventions must be observable in resolved contracts and physical resource records.

7.2 Reset, Workload, and Failure Regimes

Every transition uses a prespecified reset or washout procedure. It drains or cancels in-flight work, restores mutable data and policy snapshots, clears or versions caches and sessions, resets rate-limit and retry state, reinitializes scheduler history, and waits through a measured stabilization interval. The predeclared reset sentinel metrics and bounds are: zero prior-period contract identities in queue, cache, session, retry, rate-limit, and scheduler-state inventories; exact data, policy, model, tool, and assigned-treatment manifest hashes; a 90% confidence interval for sentinel log-p95 latency relative to a fresh-start reference wholly inside [-log(1.05),log(1.05)]; and a one-sided 95% lower confidence bound for the sentinel every-requirement satisfaction risk difference above -0.025. All conditions must pass before the next period begins.

A sequence is primary-analysis eligible only if every assigned period completes and every transition passes all reset sentinels before outcomes are unmasked. A failed transition invalidates the complete sequence; partial sequences do not enter the primary estimator. The power plan fixes a minimum number K_{min} of complete sequences per treatment order. Loss of order balance or fewer than K_{min} eligible sequences for any order makes P1 inconclusive. Reset-failure and sequence-completion rates are preregistered and reported by incoming treatment, preceding treatment, period, and order before outcome unmasking. The complete-sequence estimator is paired with an assignment-based intention-to-treat sensitivity analysis over every randomized cluster-period: reset failure counts as an operational failure for Q_{req}, missing R is bounded by prespecified worst-case values, and a tipping-point analysis varies outcomes for failed transitions. Treatment imbalance in reset failure, or a P1 decision that changes within the sensitivity range, makes P1 inconclusive. Sequence failures and partial-period outcomes remain enumerated in the report.

The workload mix, arrival process, task corpus, identity distribution, model and tokenizer, tool versions, external-service emulator or contract, and data snapshots are fixed across crossover arms. Every eligible cell is repeated over preregistered sampling seeds, arrival-process seeds, and system seeds. Failure regimes are crossed or blocked explicitly: normal operation, worker loss, delayed external service, quota pressure, and one or more injected recoupling channels. The base P1 decision and diagnostic failure-regime results are reported separately.

Pre-outcome exclusions are determined from admission, reset, instrumentation, and infrastructure-health facts before semantic or runtime outcomes are revealed. Permitted exclusions include failed reset sentinels, corrupted treatment assignment, unavailable independent instrumentation denominator, a noncompatible Scaffold class, or an external incident that violates Omega. Load-induced failure is an outcome, not an exclusion. Exclusion counts and cluster-period identities remain in the report.

7.3 Estimands and Uncertainty

The primary R(c,s) analysis estimates $Delta_R$ on the specified log-p95 scale with cluster and period structure, sequence and failure-regime terms, and prespecified workload strata. The model includes first-order lagged capability, lagged Scaffold capacity, their lagged interaction, and their preregistered interactions with current treatment. First-period lag terms are undefined and do not contribute to carryover contrasts. Differential carryover is acceptable only when every 90% confidence interval for a lag-by-current-treatment contrast on Y_R lies inside $[-m_R/2,m_R/2]$ and every one-sided 95% lower bound for the corresponding Q_{req} risk difference exceeds $-m_Q/2$. An unresolved or out-of-bound lag effect makes P1 inconclusive even after reset sentinels pass.

Cluster-aware uncertainty uses a prespecified mixed-effects model with a small-sample correction, checked by cluster-level randomization inference when the order design permits [21]. The exact covariance structure, finite-sample correction, multiplicity adjustment for more than two treatment levels, and fallback when too few clusters remain are fixed in the analysis plan. Request-level standard errors are not valid substitutes. The Q(c,s) analysis uses the Q_{req} risk differences and non-inferiority rule defined in Section 3.3; aggregate task scores remain secondary so that one strong subtask cannot hide a required failure.

The E(c,s) counterfactual is one baseline: paired safe replay in a sealed emulator. For every sampled immutable request/event trace in each (c,s) cell, the emulator runs an enforcement-on replay and a minimal pass-through replay

with identical capability and Scaffold configuration, seeds, snapshots, model and tool outputs, and safe stub effectors. The minimal pass-through arm consumes the prevalidated trace but omits the admission, mediation, isolation, and evidence operations whose runtime cost is being estimated. Replay order is randomized within each pair. Out-of-band emulator containment remains active in both arms and is excluded from runtime enforcement cost. For component k, the cell estimator is the mean paired on-minus-baseline difference $\hat{E}_k(c,s)$. The analysis reports admission, mediation, isolation, reset, evidence-processing, and total components on their preregistered natural-unit scales. Every one-sided 95% upper confidence bound must lie below its component budget $B_{E,k}$ in every required (c,s) cell. A missing pair or unresolved bound is inconclusive; a lower confidence bound above budget falsifies P1.

Power is designed around the interaction-equivalence and semantic non-inferiority margins, not the larger and easier Scaffold main effect. Equivalence and non-inferiority decisions use confidence-bound rules fixed before collection [2]. The plan states clusters, periods, observations per period, assumed intracluster correlation, carryover allowance, minimum detectable interaction, semantic non-inferiority margins, and expected attrition from pre-outcome exclusions. If the achieved interval cannot distinguish the interaction bound, the result is inconclusive even when the point estimate is near zero.

To make the power rule concrete: with a between-period log-latency standard deviation of 0.4 on the log-p95 scale, an intracluster correlation of 0.05, and 500 admitted runs per cluster-period, the variance of a single cell mean is the per-run variance times (1 + (n-1) times the correlation) divided by (k times n), and the difference-in-differences contrast quadruples that variance. The minimum detectable interaction at 80% power is then roughly 0.08 on the log scale (an 8% multiplicative interaction) only when there are about 41 clusters per treatment order; any plan with fewer clusters must be declared underpowered before collection. The numbers are illustrative, not a substitute for the preregistered variance model.

Falsification matrix: P1 is the primary release decision

Hypothesis	Intervention and control	Evidence plane	Falsification or inconclusive condition
P1 PRIMARY capability x Scaffold separability	Cluster-period randomized crossover. Independently vary activated capability configuration c and Scaffold capacity configuration s; hold workload and data snapshot fixed; preregister reset or washout.	Semantic outcome Q(c,s); runtime response R(c,s); enforcement overhead E(c,s); six condition-violation rates; cluster-aware uncertainty; capability-by-Scaffold interaction estimand.	Falsified within Omega if a semantic, runtime, or enforcement interaction exceeds its preregistered margin. Inconclusive if operating-region, condition, or detectable-interaction requirements fail. Otherwise supported within Omega.
P15	Vector gate	proposal-bank contrasts	margin failure or phantom violation
P16	Summary reconstruction	held-out joint contrast	direct-reading control wins
P17	Registry change control	dependency-change oracle	unanticipated propagation

Secondary protocols remain diagnostic; they do not substitute for the P1 release decision.

Figure 8. *Shown: P1 maps to cluster-period interventions on c and s, six evidence obligations, semantic and runtime estimands, enforcement cost, and explicit falsification or inconclusive conditions. Why it matters: the matrix ties every architectural claim to an observation and prevents missing coverage or low power from being reported as separability. Class: proposed measurement design.*

7.4 Reporting

The primary result table reports each (c,s) cell, cluster-period count, exclusion count, six coverage rates, six violation intervals, R(c,s), Q(c,s), E(c,s), the interaction estimate and interval, all preregistered margins, and the final three-way decision. Capacity-response curves remain visible even when P1 is inconclusive. This preserves the useful positive Scaffold main effect while refusing to infer independence from it.

8. Secondary Protocols

The architecture suggests narrower studies that do not enter the P1 decision unless preregistered as part of an obligation; each is stated with its refuting condition. A Harness-bypass ablation injects deliberately unbound effect calls and asks whether complete-mediation instrumentation detects them; the mediation claim is weakened if an unbound call reaches an effector without a typed rejection in the trace. An inline-expansion comparison contrasts granting authority inline with bounded derivation (Figure 4); the derivation protocol is supported only if inline expansion measurably widens the activated authority surface or the trace. An external-data reconstruction study tests whether source, snapshot, and access-decision identity can be rebuilt from the returned evidence bundle (Figure 5); it fails if an evidence bundle cannot reproduce those identities. A dry-run economics study compares planning cost against avoided failed work (Figure 6); dry-run is not justified where its predictions are poorly calibrated or its cost exceeds avoided work. A

lifecycle-transition study checks whether model, tool, and policy changes trigger the expected Skill release transitions (Figure 7); the lifecycle is insufficient if a production-significant change bypasses revalidation.

Automated Skill training, reconstructive summaries, and cross-registry change propagation are future work. They require their own interventions, oracles, and evidence boundaries. They should not be attached to P1 as additional propositions, because doing so would turn one systems question into an open-ended architecture survey.

9. Evidence Boundary and Threats

This manuscript defines a proposed architecture and experiment. It provides no completed runtime implementation, no measured enforcement budget, no dataset, no benchmark result, and no claim about a deployment's supported scale. The diagrams are responsibility and protocol views, not screenshots or implementation evidence.

P1 is conditional on the declared threat model and Omega. Unobserved shared services, model sensitivity to timing or resource metadata, imperfect resets, changing external data, and scheduler adaptation can create coupling outside the six measured channels. Strict coverage rules reduce false support but can make useful deployments inconclusive. That is an honest boundary: the paper values a defensible measurement over a universal claim.

Non-inferiority margins can also be chosen too loosely, and enforcement budgets can omit organizational or operational cost. The study must justify margins in domain terms and report every component, including evidence retention and reset work. Multiple semantic outcomes require a prespecified decision hierarchy or multiplicity rule. Diagnostic injections must be strong enough to validate monitor sensitivity without contaminating base periods.

The crossover improves efficiency when clusters can return to a comparable state. It is unsuitable for irreversible learning, persistent user adaptation, or infrastructure changes with long carryover. Such systems need parallel clusters or longer stepped designs, and the resulting evidence should not be described as this crossover protocol.

10. Conclusion

Cost-aware capability-capacity separability is a bounded systems claim. Skill owns versioned behavior; Harness owns logical admission and control; Scaffold owns physical execution and isolation; and the external data substrate owns authoritative meaning, access facts, and snapshots. Those boundaries produce testable records rather than independence by assertion.

P1 asks whether activated independently deployable behavior on admitted paths preserves the capacity-response relationship, whether compatible capacity changes preserve semantic outcomes, and whether the required controls fit an enforcement budget. A cluster-period randomized crossover, six measured obligations, explicit margins, and a three-way decision make that question answerable. The next scientific step is to implement the monitors and run the study, including the cases that end as falsified within Omega or inconclusive.

References

[1] Parnas, David L.. 1972. On the Criteria To Be Used in Decomposing Systems into Modules. Communications of the ACM. Volume 15, issue 12, pages 1053-1058. DOI: 10.1145/361598.361623. URL: https://doi.org/10.1145/361598.361623.

[2] Schuirmann, Donald J.. 1987. A Comparison of the Two One-Sided Tests Procedure and the Power Approach for Assessing the Equivalence of Average Bioavailability. Journal of Pharmacokinetics and Biopharmaceutics. Volume 15, issue 6, pages 657-680. DOI: 10.1007/BF01068419. URL: https://doi.org/10.1007/BF01068419.

[3] Piaggio, Gilda, Elbourne, Diana R., Pocock, Stuart J., Evans, Stephen J. W., Altman, Douglas G., the CONSORT Group. 2012. Reporting of Noninferiority and Equivalence Randomized Trials. JAMA. Volume 308, issue 24, pages 2594-2604. DOI: 10.1001/jama.2012.87802. URL: https://doi.org/10.1001/jama.2012.87802.

[4] Anderson, James P.. 1972. Computer Security Technology Planning Study. issue ESD-TR-73-51, Volume II. URL: https://csrc.nist.gov/csrc/media/publications/conference-paper/2001/10/01/proceedings-of-the-24th-nissc-2001/documents/papers/197.pdf.

[5] Saltzer, Jerome H., Schroeder, Michael D.. 1975. The Protection of Information in Computer Systems. Proceedings of the IEEE. Volume 63, issue 9, pages 1278-1308. DOI: 10.1109/PROC.1975.9939. URL: https://doi.org/10.1109/PROC.1975.9939.

[6] Miller, Mark S., Yee, Ka-Ping, Shapiro, Jonathan S.. 2003. Capability Myths Demolished. issue SRL2003-02. URL: https://srl.cs.jhu.edu/pubs/SRL2003-02.pdf.

[7] Levin, Roy, Cohen, Ellis S., Corwin, William M., Pollack, Fred J., Wulf, William A.. 1975. Policy/Mechanism Separation in Hydra. Proceedings of the Fifth ACM Symposium on Operating Systems Principles. pages 132-140. DOI: 10.1145/800213.806531. URL: https://doi.org/10.1145/800213.806531.

[8] Kreutz, Diego, Ramos, Fernando M. V., Verissimo, Paulo E., Rothenberg, Christian E., Azodolmolky, Siamak, Uhlig, Steve. 2015. Software-Defined Networking: A Comprehensive Survey. Proceedings of the IEEE. Volume 103, issue 1, pages 14-76. DOI: 10.1109/JPROC.2014.2371999. URL: https://doi.org/10.1109/JPROC.2014.2371999.

[9] Goguen, Joseph A., Meseguer, Jose. 1982. Security Policies and Security Models. Proceedings of the 1982 IEEE Symposium on Security and Privacy. pages 11-20. DOI: 10.1109/SP.1982.10014. URL: https://doi.org/10.1109/SP.1982.10014.

[10] Kephart, Jeffrey O., Chess, David M.. 2003. The Vision of Autonomic Computing. Computer. Volume 36, issue 1, pages 41-50. DOI: 10.1109/MC.2003.1160055. URL: https://doi.org/10.1109/MC.2003.1160055.

[11] IBM Corporation. 2006. An Architectural Blueprint for Autonomic Computing. URL: https://www.ibm.com/autonomic/pdfs/AC_Blueprint_White_Paper_4th.pdf.

[12] International Organization for Standardization, International Electrotechnical Commission. 2023. Systems and Software Engineering - Systems and Software Quality Requirements and Evaluation (SQuaRE) - Product Quality Model. issue ISO/IEC 25010:2023. URL: https://www.iso.org/standard/78176.html.

[13] Kleinrock, Leonard. 1975. Queueing Systems, Volume 1: Theory.

[14] Mars, Jason, Tang, Lingjia. 2011. Bubble-Up: Increasing Utilization in Modern Warehouse Scale Computers via Sensible Co-Locations. Proceedings of the 44th Annual IEEE/ACM International Symposium on Microarchitecture. pages 248-259. DOI: 10.1145/2155620.2155650. URL: https://doi.org/10.1145/2155620.2155650.

[15] Dean, Jeffrey, Barroso, Luiz Andre. 2013. The Tail at Scale. Communications of the ACM. Volume 56, issue 2, pages 74-80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[16] Tallam, Krti. 2026. A Five-Plane Reference Architecture for Runtime Governance of Production AI Agents. arXiv preprint arXiv:2606.12320. URL: https://arxiv.org/abs/2606.12320.

[17] Xie, Yi, Du, Jiawei, Cheng, Yu, Zhou, Jiuan, Yin, Zhaoxia. 2026. Benign in Isolation, Harmful in Composition: Security Risks in Agent Skill Ecosystems. arXiv preprint arXiv:2606.15242. URL: https://arxiv.org/abs/2606.15242.

[18] Soni, Deepak. 2026. Falsifiable Release Gates for Self-Improving Systems: Standing Invariants at Scale. arXiv preprint arXiv:2607.13070. URL: https://arxiv.org/abs/2607.13070.

[19] Fan, Haodi, Lan, Zucong. 2026. Skillware: A Software Ontology and Engineering Lifecycle for Persistent Behavioral Artifacts. arXiv preprint arXiv:2607.18970. URL: https://arxiv.org/abs/2607.18970.

[20] Jones, Byron, Kenward, Michael G.. 2014. Design and Analysis of Cross-Over Trials. DOI: 10.1201/b17537. URL: https://doi.org/10.1201/b17537.

[21] Turner, Rebecca M., White, Ian R., Croudace, Tim. 2007. Analysis of Cluster Randomized Cross-Over Trial Data: A Comparison of Methods. Statistics in Medicine. Volume 26, issue 2, pages 274-289. DOI: 10.1002/sim.2537. URL: https://doi.org/10.1002/sim.2537.